

Quiz — 2.2.1 Programming Techniques

Name: _____ Date: _ **Mark:** ____ / 25

OCR H446 · Component 02 · 2.2.1

Section A — Multiple choice (8 marks)

A1. Which loop is guaranteed to execute its body **at least once**? A. `while` B. `for...next` C. `do...until` D. `if...else` Your answer: ☐

A2. What is the defining difference between a function and a procedure? A. A function can take parameters; a procedure cannot B. A function returns a single value; a procedure does not return a value C. A procedure is recursive; a function is iterative D. A function is global; a procedure is local Your answer: ☐

A3. A recursive subroutine with no reachable base case will most likely cause: A. A syntax error B. An infinite `while` loop C. A stack overflow D. A logic error that returns 0 Your answer: ☐

A4. When an argument is passed **by reference**, what is passed to the subroutine? A. A copy of the value B. The memory address of the original variable C. The data type only D. A new global variable Your answer: ☐

A5. A variable declared inside a subroutine is: A. Global, visible everywhere B. Local, visible only within that subroutine C. A constant D. Automatically passed by reference Your answer: ☐

A6. Which OOP concept keeps attributes **private** and provides public methods to access them? A. Inheritance B. Polymorphism C. Encapsulation D. Overloading Your answer: ☐

A7. A subclass replaces an inherited method with one of the **same name and parameters**. This is: A. Overloading B. Overriding C. Instantiation D. Encapsulation Your answer: ☐

A8. Which IDE feature lets a programmer **pause execution at a chosen line** to inspect variables? A. Syntax highlighting B. Auto-indentation C. Breakpoint D. Code completion Your answer: ☐

Section B — Short answer (12 marks)

B1. State the **two** parts every correctly written recursive subroutine must contain, and give the role of each. [2]

B2. Consider the program below. State the value printed and explain why, referring to **scope**. [3]

```

total = 10

procedure update()
    total = 5
    print(total)
endprocedure

update()
print(total)

```

Note: in this language an assignment to `total` inside the procedure creates a **new local** variable `total`; it does not change the global.

Output line 1: __ **Output line 2:** ____

Explanation: _____

B3. A subroutine is called with `x = 3`. Explain what the **final value of x** is after each call below, and why. **[3]**

```

procedure addOneByVal(byVal n)
    n = n + 1
endprocedure

procedure addOneByRef(byRef n)
    n = n + 1
endprocedure

```

After `addOneByVal(x)` : `x =` __ **because** _____

After `addOneByRef(x)` : `x =` __ **because** _____

B4. Trace the call `factorial(4)` for the routine below. Show the value returned at each level and the final result. **[2]**

```

function factorial(n)
    if n <= 1 then
        return 1
    else
        return n * factorial(n - 1)
    endif
endfunction

```

B5. Explain **one** advantage of preferring **local** variables over global variables. **[1]**

B6. Define the terms **class** and **object**, making clear how they relate. **[1]**

Section C — Exam-style question (5 marks)

A `BankAccount` class is being written. Complete the OCR-style class so that it:

- has a **private** attribute `balance` initialised to 0 by the constructor,
- has a method `deposit(amount)` that adds `amount` to the balance,
- has a method `getBalance()` that returns the current balance.

```
class BankAccount
  private balance

  public procedure new()
    // (a) initialise balance

  endprocedure

  // (b) deposit method

  // (c) getBalance method

endclass
```

Write your answers for (a), (b) and (c) below. **[5]**

Answer key

Section A (1 mark each = 8)

Q	Ans	Why
A1	C	<code>do...until</code> is post-conditioned — the condition is tested at the end , so the body always runs at least once. <code>while</code> may run zero times.
A2	B	The defining difference: a function returns a single value (usable in an expression); a procedure returns none.
A3	C	With no reachable base case, calls never stop; each pushes a stack frame until memory is exhausted → stack overflow .
A4	B	By reference passes the memory address of the original, so changes affect the original.
A5	B	A variable declared inside a subroutine is local — visible only within it.
A6	C	Encapsulation = bundling data + methods with private attributes accessed via public methods (data hiding).
A7	B	Same name and parameters replacing an inherited method = overriding (overloading = same name, different parameters).
A8	C	A breakpoint pauses execution at a chosen line so variables can be inspected.

Section B (12)

B1. [2] - Base case — the condition that stops the recursion, with no further recursive call (1). - **General / recursive case** — calls the subroutine itself, moving closer to the base case (1).

B2. [3] - Output line 1: **5** (1) — inside the procedure the local `total` holds 5. - Output line 2: **10** (1) — outside, the global `total` is unchanged. - Explanation (1): the assignment inside `update()` creates/uses a **local** variable `total` whose scope is the procedure only; it **shadows** the global but does not alter it, so after the call the global is still 10.

B3. [3] - After `addOneByVal(x) : x = 3` (1) because a **copy** of the value is passed; `n` is incremented but the original `x` is **unchanged** (1 for either by-value point). - After `addOneByRef(x) : x = 4` (1) because the **memory address** of `x` is passed, so updating `n` updates the original `x`.

(Mark points: by value → original unchanged; by reference → original can be / is changed. The *consequence* is the mark, not just "a copy is made".)

B4. [2] Trace of `factorial(4)` (frames pushed then popped, LIFO):

```
factorial(4) = 4 * factorial(3)
factorial(3) = 3 * factorial(2)
factorial(2) = 2 * factorial(1)
factorial(1) = 1                (base case, n <= 1)
```

Unwinding: $2! = 2 \rightarrow 3! = 6 \rightarrow 4! = 24$. - 1 mark for correct working/returns at each level; 1 mark for final answer 24*.

B5. [1] Any one: fewer side effects / no accidental change elsewhere; memory freed when the subroutine exits; more modular / reusable / easier to debug and maintain.

B6. [1] A **class** is a template/blueprint defining the attributes and methods; an **object** is a specific **instance** of that class created at run time. (Both halves needed for the mark.)

Section C (5)

Model answer:

```
class BankAccount
  private balance

  public procedure new()
    balance = 0           // (a)
  endprocedure

  public procedure deposit(amount) // (b)
    balance = balance + amount
  endprocedure

  public function getBalance()      // (c)
    return balance
  endfunction
endclass
```

Mark points (1 each, max 5): 1. Constructor sets `balance = 0`. 2. `deposit` declared with a parameter `amount` (procedure). 3. `deposit` body adds `amount` to `balance` (`balance = balance + amount`). 4. `getBalance` declared as a **function** (it returns a value). 5. `getBalance` body uses `return balance`.

(Accept correct OCR-style syntax variants, e.g. `public function/public procedure`. Using a function for `deposit` or a procedure for `getBalance` loses the relevant point since a value must be returned by `getBalance`.)

Total: 8 + 12 + 5 = 25.